Write a program that prints a simple chessboard.

Input format:
The first line contains the number of inputs T. The lines after that contain a different values for size of the chessboard

Output format:
Print a chessboard of dimensions size * size. Print a Print W for white spaces and B for black spaces.

Input:
2
3
5

Output:

WBW
BWB
WBW
WBWBW
BWBWB
WBWBW
BWBWB
WBWBW

```c
#include<stdio.h>
int main()
{
    int a,b,c,size;
    scanf("%d",&a);
    for(int i=0;i<a;i++)
    {
        scanf("%d",&size);
    for(b=1;b<=size;b++)
      {
        for(c=1;c<=size;c++)
        {
            if((b+c)%2==0)
            {
                printf("W");
            }
            else
            {
                printf("B");
            }
        }
        printf("\n");
      }
    }
}
```

	Input	Expected	Got	
✓	2 3 5	WBW BWB WBW WBWBW BWBWB WBWBW BWBWB WBWBW	WBW BWB WBW WBWBW BWBWB WBWBW BWBWB WBWBW	✓

Passed all tests! ✓

Let’s print a chessboard!

Write a program that takes input:

The first line contains T, the number of test cases Each test case contains an integer N and also the starting character of the chessboard

Output Format
Print the chessboard as per the given examples

Sample Input / Output

Input:
2
2 W
3 B

Output:
WB
BW
BWB
WBW
BWB

```c
#include<stdio.h>
int main()
{
  int t,size;
  char start,other;
  scanf("%d",&t);
  while(t--)
  {
      scanf("%d %c",&size,&start);
      other=(start=='W')?'B':'W';
      for(int i=0;i<size;i++)
      {
          for(int j=0;j<size;j++)
          {
              printf("%c",(i+j)%2==0?start:other);
          }
          printf("\n");
      }
  }
}
```

	Input	Expected	Got	
✓	2 2 W 3 B	WB BW BWB WBW BWB	WB BW BWB WBW BWB	✓

Passed all tests! ✓

Decode the logic and print the Pattern that corresponds to given input.

If N= 3 then pattern will be

10203010011012
**4050809
****607

If N= 4, then pattern will be:

1020304017018019020
**50607014015016
****809012013
******10011

Constraints
2 <= N <= 100

Input Format
First line contains T, the number of test cases
Each test case contains a single integer N

Output
First line print Case #i where i is the test case number
In the subsequent line, print the pattern

Test Case 1
3
3
4
5

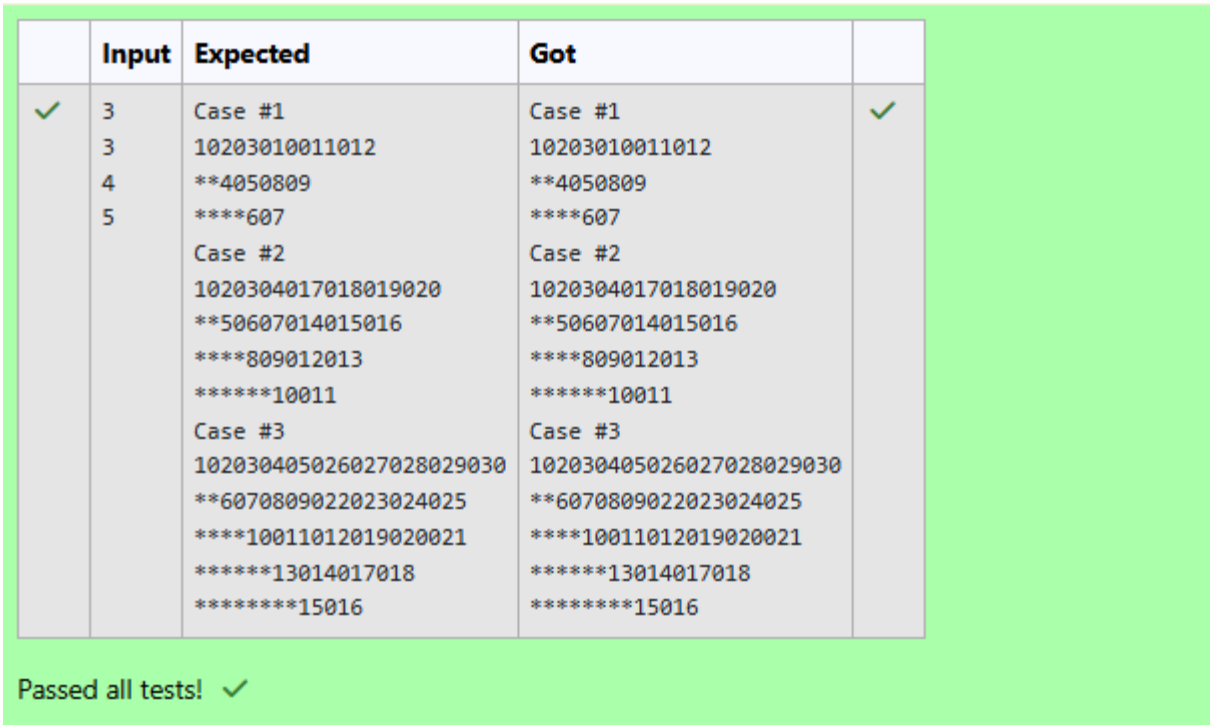
Output

Case #1
10203010011012
**4050809
****607

Case #2
1020304017018019020
**50607014015016
****809012013
******10011

Case #3
10203040502602702802 9030
**6070809022023024025
****10011012019020021
******13014017018
********15016

```c
#include<stdio.h>
int main()
{
    int T;
    scanf("%d",&T);
    for(int x=1;x<=T;x++)
    {
        printf("Case #%d\n",x);
        int n;
        scanf("%d",&n);
        int f=1,b=n*(n+1);
        for(int i=0;i<n;i++)
        {
            for(int k=0;k<2*i;k++)
            {
                printf("*");
            }
            printf("%d",f);
            f++;
            for(int j=2;j<=n-i;j++)
            {
                printf("0%d",f);
                f++;
            }
            for(int l=b-(n-i)+1;l<=b;l++)
            {
                printf("0%d",l);
            }
            b-=n-i;
            printf("\n");
        }
    }
}
```



	Input	Expected	Got	
✓	3 3 4 5	Case #1 10203010011012 **4050809 ****607 Case #2 1020304017018019020 **50607014015016 ****809012013 ******10011 Case #3 10203040502602702802903 0 **6070809022023024025 ****10011012019020021 ******13014017018 ********15016	Case #1 10203010011012 **4050809 ****607 Case #2 1020304017018019020 **50607014015016 ****809012013 ******10011 Case #3 10203040502602702802903 0 **6070809022023024025 ****10011012019020021 ******13014017018 ********15016	✓

Passed all tests! ✓

The k-digit number N is an Armstrong number if and only if the k-th power of each digit sums to N

Given a positive integer N, return true if and only if it is an Armstrong number.

Example 1:

Input:
153

Output:
true

Explanation:
153 is a 3-digit number, and 153 = 1^3 + 5^3 + 3^3.

Example 2:

Input:
123

Output:
false

Explanation:

123 is a 3-digit number, and 123 != 1^3 + 2^3 + 3^3 = 36.

Example 3:

Input:
1634

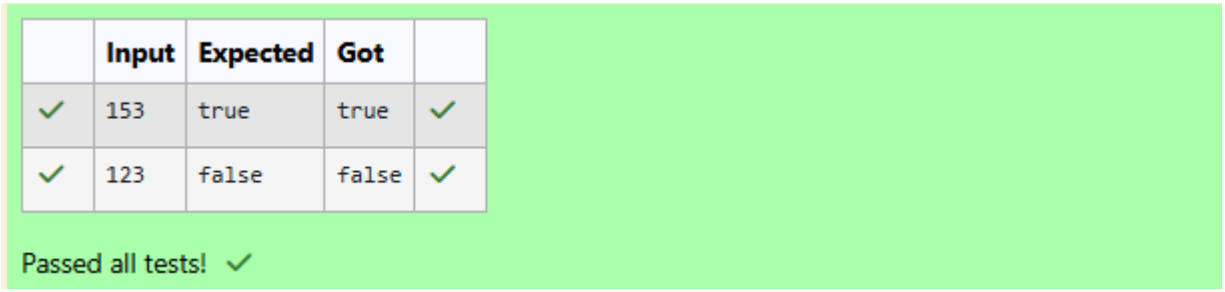
Output:
true

Note: 1 <= N <= 10^8

```c
#include<stdio.h>
#include<math.h>
int main()
{
    int a,count,b,c,rem1,arm;
    scanf("%d",&a);
    b=a;
    while(b!=0)
    {
        count++;
        b=b/10;
    }
    c=a;
    while(c!=0)
    {
        rem1=c%10;
        arm=arm+(pow(rem1,count));
        c=c/10;
    }
    if(a==arm)
    {
        printf("true");
    }
    else
    {
        printf("false");
    }

}

```



	Input	Expected	Got	
✓	153	true	true	✓
✓	123	false	false	✓

Passed all tests! ✓

Take a number, reverse it and add it to the original number until the obtained number is a palindrome.

Constraints 1<=num<=99999999

Sample Input 1
32

Sample Output 1
55

Sample Input 2
789

Sample Output 2
66066 .

```c
#include<stdio.h>
int main()
{
    long long int num, sum, rev, a,b;
    scanf("%lld",&num);
    while(1)
    {
        rev=0;
        b=num;
        while(num)
        {
            rev=rev*10+(num%10);
            num=num/10;
        }
        sum = b+rev;
        a=sum;
        rev=0;
        while(sum)
        {
            rev=rev*10+(sum%10);
            sum/=10;
        }
        if(a==rev)
        break;
        num=a;
    }

         printf("%lld",a);
}
```

	Input	Expected	Got	
✓	32	55	55	✓
✓	789	66066	66066	✓

Passed all tests! ✓

A number is considered lucky if it contains either 3 or 4 or 3 and 4 both in it. Write a program to print the nth lucky number. Example, 1st lucky number is 3, and 2nd lucky number is 4 and 3rd lucky number is 33 and 4th lucky number is 34 and so on. Note that 13, 40 etc., are not lucky as they have other numbers in it.

The program should accept a number 'n' as input and display the nth lucky number as output.

Sample Input 1:
3

Sample Output 1:
33

Explanation:
Here the lucky numbers are 3, 4, 33, 34., and the 3rd lucky number is 33.

Sample Input 2:
34

Sample Output 2:
33344

```c
#include<stdio.h>
int main()
{
    long int i,j;
    int rem,n,count=0,a;
    scanf("%d",&n);
    for(i=1;count<=n;i++)
    {
        a=0;
        j=i;
        while(j>0)
        {
            rem=j%10;
            if(rem==3||rem==4)
            j=j/10;
            else
            {
                a=1;
                break;
            }
        }
        if(a==0)
        {
            count++;
            if(count==n)
            break;
        }
    }
    printf("%ld",i);

}
```

	Input	Expected	Got	
✓	34	33344	33344	✓

Passed all tests! ✓